

Analytics dashboard for an AI chat app

front: what to ask, what to design · one output topic, three consumers · erasure by shredding a key

Anthropic questions this is: SD Q5 data infrastructure ("ingest, process and serve large-scale data to stakeholders with varying needs and sensitivities"; last asked Sep 7, 2026) in the product-facing shape **hellointerview** reports as "analytics dashboard for an AI chat app" (senior, data-infra loop, mid Mar 2026; also Salesforce, Airbnb) · same pipeline as the **SD Q5 telemetry variant** (prachub Sep 18 onsite: schema registry, inconsistent names, 10× client, dedupe retries) · Rik's Q10 unified telemetry feeds the GPU panel · igotanoffer Q1 chat service is the thing being observed. **Anthropic adds:** access control by sensitivity (row/column policies, audit), infrastructure per facet, failure and growth (bad data blast radius, backfill, cardinality).

1 REQUIREMENTS 0–5 min · personas, then numbers

- **Ask:** three metrics per persona? freshness per persona (seconds health, minutes behaviour, daily business)? DAU and messages per user? may we store chat content (no)? retention: warm 360 d, cold beyond? GDPR erasure SLA, HIPAA for Support?
- **Engineering:** per-model p50/p99, error rate per minute, model usage, queue depth, GPU util; alerts. **Product:** DAU/WAU, session length, retention, thumbs, revenue by plan. **Support:** per-user timeline of sessions and errors, never content.
- **Technical challenges:** isolation (analytics never degrades chat) · freshness 1–5 s, <50 ms only on the hot query path · availability independent of chat · privacy: no content, tokenised identity, erasure keeps aggregates · cardinality · retention cost.
- **Anchors:** 30M DAU × 20 msgs × 5 events ≈ 3B/day ≈ 35k/s avg, 150k/s peak · 300 B → 1 TB/day raw · warm 360 d ≈ 360 TB raw, rollups GBs/day.

2 CORE ENTITIES 5–9 min · the event record is the answer to homework 3

- **Event**(event_id UUIDv7, type, schema_version, ts_event, ts_ingest, source, session_id, user_token, request_id, model, region, plan, client_version, measures{latency_ms, tokens_in/out, status}, attributes{}, sensitivity tags) — immutable source of truth; fixed envelope, per-type attributes
- **Session**(session_id, user_token, start, end, client, origin)
- **PrivacyToken**(user_id → token, per-user key, created_at, revoked_at); analytics never stores user_id
- **Rollup**(metric, window 1s|1m|1h|1d, dims, count, sum, p50, p99 sketch, watermark)
- **Dashboard**(audience, widgets[]) · **AlertRule**(metric, condition, window, group_by, tier nrt|fast|slow, channel, escalation)
- Not entities: the chat message (never stored), the GPU (a dimension).

3 API 9–13 min · one endpoint per need

- **POST /v1/events** batch, fire-and-forget, always 202, idempotent on event_id
- **GET /v1/metrics?metric&dims&window&from&to** → cached rollups
- **GET /v1/sessions/{id}/timeline** (Support, RBAC, no content fields, audited)
- **POST /v1/dashboards** · **POST /v1/alerts**
- **DELETE /v1/users/{id}/privacy-token** = GDPR erasure (shred the key)
- internal: **GET /schemas/{type}/{version}**; evaluators read rollups, write notifications

4 DATA FLOW 13–20 min · one event, end to end

- service emits event with the user's token → **collector** buffers 5–10 ms, batches; backpressure drops analytics, never the request →
- **ingest topic** (Kafka, by session_id, acks=1) → **stream processor**: schema check, dedupe on event_id, 1-s and 1-min rollups with p99 sketches, watermarks → **one output topic** →
- **three consumers:** real-time → dashboard API → WebSocket push · DB writer → hot store (filter, sort, history) · cold writer → lake (raw from the ingest topic + rollups; warm 360 d, cold beyond) →
- **alert evaluator** per tier reads rollups and routes. **Erasure:** delete the token key; rows become unattributable, aggregates unchanged.

5 HIGH-LEVEL DESIGN 20–33 min · pipeline first, stores second

- Row 1: sources → collector → ingest topic → stream processor; registry and token vault above.
- Row 2: **output topic** → **real-time consumer** → **dashboard API** → **three dashboards** (Support timeline behind RBAC).
- Row 3: DB writer → hot store; cold writer → lake; alert evaluator per tier; compliance module scanning the lake.
- **Why one topic, three consumers (Tyler):** one seam; each sink at its own pace, replayable; a slow lake write never backs up the processor; a new sink is a new consumer group.
- **Homework 2, real time:** ingest = client batching with a 5–10 ms flush, gRPC/HTTP2 to a same-region collector, ring buffer, Kafka acks=1, no sync validation; serve = in-memory rollups, incremental windows, materialised views, WebSocket push. **Honest:** 1–5 s end to end; <50 ms is the cached rollup query.

6 DEEP DIVES 33–48 min · Tyler's five, then Anthropic's extras

- **1 GDPR:** regex scrub (brittle) · global-salt hash (re-identifiable, not erasable) · delete rows (kills trends) · **per-user token + crypto-shredding:** HMAC(user_id, per-user key), erasure = delete the key; goal is non-attributability, not insider-proof. No content ever; sensitivity tags; retention tiers; compliance scanner.
- **3 Event record:** envelope (ids, both timestamps, source, schema version, correlation ids) + low-cardinality dims + numeric measures + per-type attributes; CloudEvents / OpenTelemetry shape; forward-compatible readers.
- **4 Alerts:** rules over rollups on a tier-matched schedule; group by dims; thresholds + baselines; dedupe, grouping, silences; severity, escalation; route to pager / chat / email / ticket.
- **5 Tiers:** <10 s streaming on 1-s rollups → pager (p99, error rate per model) · <60 s 1-min rollups every 15 s → chat/on-call (GPU util, queue depth) · <1 day batch on the lake → email/ticket (retention, revenue drift, cost). One evaluator per tier.
- **Anthropic:** sensitivity classes with row/column policy + audit; workload isolation; bad-data quarantine + re-run; cardinality budget. **Bow:** events are truth, emitted fire-and-forget; roll up early; one log, a hot store, a lake, an evaluator per tier; identity is a shreddable token.

The board at minute thirty

back: the pipeline, as it should look on the shared screen

